



Handling Errors

- An error is a condition that causes a disruption in the execution of a program. An error can be a result of circumstances, such as incorrect code or insufficient memory resources. Errors in a Java program are categorized into two types, compile-time errors and run-time errors.
- The compile-time errors occur when the syntax of a programming language are not followed. For example, in Java, if you use a keyword name as a variable name, a compile-time error will be raised by the compiler.
- The run-time errors occur during the execution of a program. For example, if the program runs out of memory, it results in a run-time error. Such an error is known as an exception. An exception can lead to the termination of an application. Dealing with such abnormal behavior is called **exception handling**.

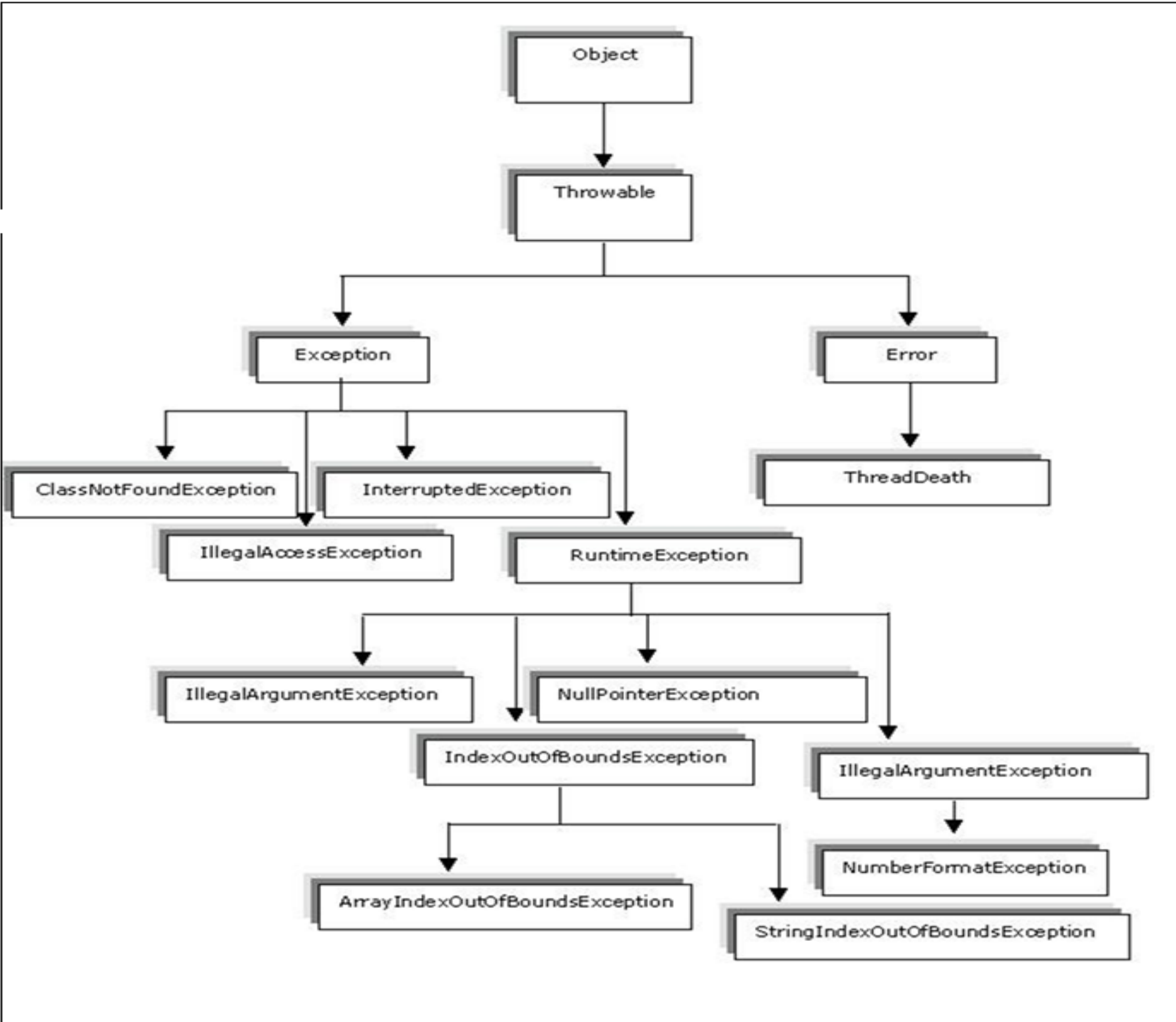
Handling Exceptions

- The term exception in Java indicates an abnormal event that occurs during the program execution and disrupts the normal flow of instructions. For example, if you divide a number by zero or open a file that does not exist, an exception is raised.
- In Java, exceptions can be handled either by the Java run-time system or by a user-defined code. Java supports various built-in classes and exception handlers to deal with exceptions.

Exploring Exceptions

- When a run-time error occurs, an exception is thrown by the JVM which can be handled by an appropriate exception handler. The Java run-time system proceeds with the normal execution of the program after an exception is handled. If no appropriate exception handler is found by the JVM, the program is terminated.
- There are several built-in exceptions that have been identified in Java. In order to deal with these exceptions, Java has various built-in exception classes.

These built-in classes are organized in a hierarchical manner, as shown in the following figure.



The Exception Hierarchy



The Throwable Class

The Throwable class is the base class of exceptions in Java. You can throw only those exception objects that are derived from the Throwable class. The following two classes are derived for the Throwable class:

- ☐ Exception
- ☐ Error

The Exception Class

The Exception class represents the conditions that a program should handle. The Exception class has various subclasses, such as `ClassNotFoundException`, `IllegalAccessException`, and `RuntimeException`. The `ClassNotFoundException` exception is thrown when a class is being referred, but no definition for the same is found. The `IllegalAccessException` exception is thrown when a particular method is not found.



The Error Class

The Error class defines the exceptions related to the Java run-time environment.

For example,

`OutOfMemoryError` is an error that occurs when there is insufficient system memory to execute a program.

Identifying Checked and Unchecked Exceptions

Java exceptions are categorized into the following types:

- ☐ Checked exceptions
- ☐ Unchecked exceptions

Checked Exceptions

These are the invalid conditions that occur in a Java program due to the problems, such as accessing a file that does not exist or referring a class that does not exist. The checked exceptions are the objects of the Exception class or any of its subclasses excluding the RuntimeException and Error classes.

For example,
a checked exception `IOException` is raised, when a program tries to read from a file that does not exist. The checked exceptions make it mandatory for a programmer to handle them. If a checked exception is not handled, then a compile-time error occurs.

The following table lists the most commonly used checked exceptions.

<i>Exception</i>	<i>Cause of Creation</i>
<code>ClassNotFoundException</code>	<i>Is thrown when the Java run-time system is unable to find the referred class.</i>
<code>IllegalAccessException</code>	<i>Is thrown when you refer an object, class, variable, constructor, or a method that is not accessible.</i>
<code>InstantiationException</code>	<i>Is thrown when you try to create an instance of a class by using the <u><code>newInstance()</code></u> method, but the referred class cannot be instantiated.</i>
<code>NoSuchMethodException</code>	<i>Is thrown when a particular method cannot be found.</i>

The Checked Exceptions in Java

Unchecked Exceptions

The unchecked exceptions occur because of programming errors. Therefore, the compiler does not force a programmer to handle these exceptions. Such errors should be handled by writing bug free codes. However, there are times such situations cannot be eradicated.

For example in a calculator application, if you divide a number by zero, an unchecked exception is raised.

The following table lists the various unchecked exceptions and their causes of creation.

<i>Exception</i>	<i>Cause of Creation</i>
<i>ArithmeticException</i>	<i>Occurs when you make an arithmetic error, such as dividing a number by zero.</i>
<i>ArrayIndexOutOfBoundsException</i>	<i>Occurs when an attempt is made to access an array element beyond the index of the array.</i>
<i>ArrayStoreException</i>	<i>Occurs when you assign an element to an array that is not compatible with type of data that can be stored in that array.</i>
<i>ClassCastException</i>	<i>Occurs when you assign a reference variable of a class to an incompatible reference variable of another class.</i>
<i>IllegalArgumentException</i>	<i>Occurs when you pass an argument of incompatible data type to a method.</i>
<i>NegativeArraySizeException</i>	<i>Occurs when you create an array with a negative size.</i>
<i>NullPointerException</i>	<i>Occurs when an application tries to use an object without allocating memory to it or calls a method of a null object.</i>
<i>NumberFormatException</i>	<i>Occurs when you want to convert a string in an incorrect format to a numeric format.</i>

Implementing Exception Handling

- When an unexpected error occurs, Java creates an exception object. After creating the exception object, Java sends it to the program by throwing the exception.
- The exception object contains information about the type of error and the state of the program when the exception occurred.
- You need to handle the exception by using an exception handler and processing the exception.
- You can implement exception handling in a program by using the following keywords and blocks:

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Using try and catch Blocks

- A try block encloses the statements that might raise an exception and defines one or more exception handlers associated with it.
- If an exception is raised within the try block, the appropriate exception handler that is associated with the try block processes the exception.
- In Java, the catch block is used as an exception handler. A try block must have at least one catch block that follows the try block, immediately.
- The catch block specifies the exception type that you need to catch.

You can declare the try and the catch block by using the following syntax:



```
try
{
    // Statements that can cause an exception.
}

catch(exceptionname obj)
{
    // Error handling code.
}
```

In the preceding syntax, the catch block accepts the object of the Throwable class or its subclass that refers to the exception caught, as a parameter. When the exception is caught, the statements within the catch block are executed.

Control Flow inside try-catch():

```
try
{
    Java Statement-1;
    Java Statement-2;
    Java Statement-3;
}
catch(exceptionname obj)
{
    Java Statement-4;
}
```

Java Statement-5;

Case 1: If there is no exception raised

Case 2: If an exception raised at Java Statement-2 and corresponding catch block matched

Case 3: If an exception raised at Java Statement-2 and corresponding catch block not matched

Case 4: If an exception raised at Java Statement-4 or Java Statement-5.

Consider a scenario in which you need to accept two integers from a user, performs their addition, and displays the result. For this, you can use the following code:

```
import java.util.Scanner;
```

```
public class Addition
```

```
{  
    public static void main(String[] args)
```

```
    {  
        int num1, num2, result;
```

```
        Scanner obj1 = new Scanner(System.in);
```

```
        System.out.println("Enter the 1st number");
```

```
        num1 = obj1.nextInt();
```

```
        System.out.println("Enter the 2nd number");
```

```
        num2 = obj1.nextInt();
```

```
        result = num1+num2;
```

```
        System.out.println("The result is "+result);
```

```
    }
```

```
}
```

While executing the preceding code, if the user provides an input other than an integer value, an exception is raised and the program terminates abnormally. To overcome this, you can implement exception handling in the preceding code by using the try-catch block as shown in the following code:



```
import java.util.Scanner;
public class Addition
{
    public static void main(String[] args)
    {
        int num1, num2, result;
        String snum1, snum2;
        Scanner obj1 = new Scanner(System.in);
        try
        { // monitor the try block for exceptions
            System.out.println("Enter the 1st number");
            snum1 = obj1.next();
            System.out.println("Enter the 2nd number");
            snum2 = obj1.next();
            num1=Integer.parseInt(snum1);
            num2=Integer.parseInt(snum2);
            result = num1+num2;
            System.out.println("The result is "+result);
        }
        catch(Exception e) /*the catch block catches the exception raised in the
try block and handles it*/
        {
            System.out.println("Please input only numeric values..!!");
        }
    } //End of Main
} //End of Class
```

In the preceding code, the try block is used to monitor the user input. If the user enters an input that is not an integer value, such as a char or a string, an exception is raised. However, the exception is successfully handled by the catch block that catches the exception raised in the try block.

Once the exception is caught, the message, **Please input only numeric values..!!** is displayed and the abnormal termination of the program is handled, successfully.

A try block can have multiple catch blocks. This is required when a code within the try block is capable of raising multiple exceptions.

You can declare multiple catch blocks with a single try statement by using the following code snippet:

```
try
{
// statements
}
catch(exceptionname1 obj1)
{
//statements to handle the exception
}
catch(<exceptionname2 obj2)
{
//statements to handle the exception
}
catch(exceptionnameN objN)
{
//statements to handle the exception
}
```

In the preceding code snippet, when the statements in the try block raise an exception, the matching catch block corresponding to the raised exception is executed. Further, the execution continues after the try-catch block.

While working with multiple catch statements, it is important to follow the exception hierarchy, such that the subclasses must appear prior to the super classes.



If the exception hierarchy is not followed, a compile-time error is generated.

Consider the following code:

```
public class Division
{
    public static void main(String[] args)
    {
        int num1, num2, result;
        Scanner obj1 = new Scanner(System.in);
        try
        { // monitor the try block for exceptions
            System.out.println("Enter the 1st number");
            num1 = obj1.nextInt();
            System.out.println("Enter the 2nd number");
            num2 = obj1.nextInt();
            result = num1 / num2;
            System.out.println("The result is " + result);
        }
    }
}
```

```
catch (Exception e)
{
    System.out.println("Please input only numeric
values...!!");
}
catch (ArithmeticException e)
{
    System.out.println("Division performed by zero...");
}
}
}
```

The preceding code will generate a compile-time error as the exception hierarchy is not followed. Therefore, in the preceding code `ArithmeticException` should be caught before `Exception`.

Java Exception Handling Example

```
public class JavaExceptionExample
{
    public static void main(String args[])
    {
        try
        {
            //code that may raise exception
            int data=100/0;
        }
        catch(ArithmeticException e)
        {
            System.out.println(e);
        }
        //rest code of the program
        System.out.println("rest of the code...");
    }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero

rest of the code...

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where `ArithmeticException` occurs

If we divide any number by zero, there occurs an `ArithmeticException`.

```
int a=50/0;//ArithmeticException
```

2) A scenario where `NullPointerException` occurs

If we have a null value in any variable, performing any operation on the variable throws a `NullPointerException`.

```
String s=null;  
System.out.println(s.length());//NullPointerException
```

Common Scenarios of Java Exceptions

3) A scenario where `NumberFormatException` occurs

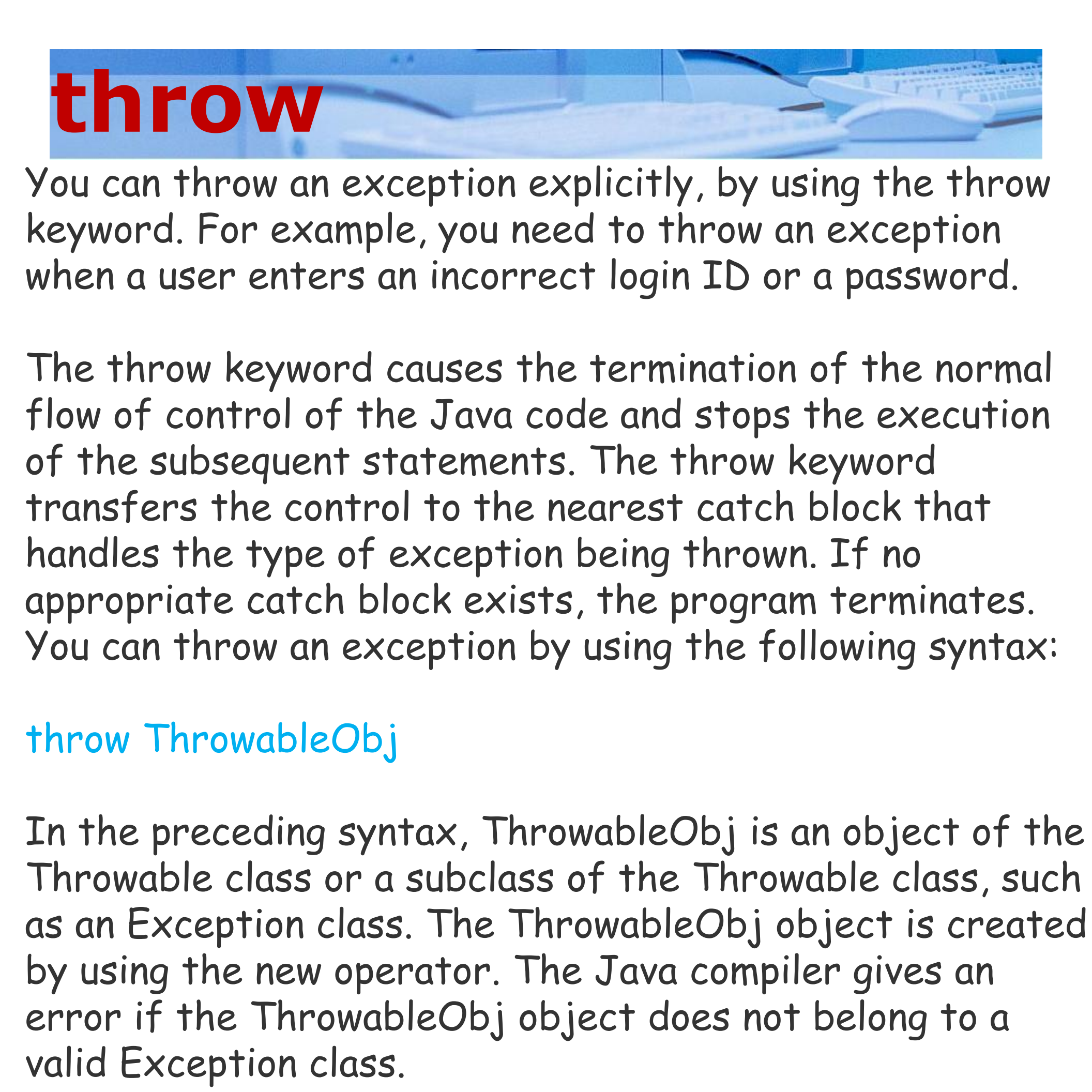
If the formatting of any variable or number is mismatched, it may result into `NumberFormatException`. Suppose we have a string variable that has characters; converting this variable into digit will cause `NumberFormatException`.

```
String s="abc";  
int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

When an array exceeds to it's size, the `ArrayIndexOutOfBoundsException` occurs. there may be other reasons to occur `ArrayIndexOutOfBoundsException`. Consider the following statements.

```
int a[]=new int[5];  
a[10]=50; //ArrayIndexOutOfBoundsException
```



throw

You can throw an exception explicitly, by using the throw keyword. For example, you need to throw an exception when a user enters an incorrect login ID or a password.

The throw keyword causes the termination of the normal flow of control of the Java code and stops the execution of the subsequent statements. The throw keyword transfers the control to the nearest catch block that handles the type of exception being thrown. If no appropriate catch block exists, the program terminates. You can throw an exception by using the following syntax:

throw ThrowableObj

In the preceding syntax, ThrowableObj is an object of the Throwable class or a subclass of the Throwable class, such as an Exception class. The ThrowableObj object is created by using the new operator. The Java compiler gives an error if the ThrowableObj object does not belong to a valid Exception class.

Consider the following code that demonstrates the implementation of the throw statement:



```
public class ThrowDemo
{
    void display()
    {
        throw new RuntimeException();
    }
    public static void main(String[] args)
    {
        ThrowDemo obj1=new ThrowDemo();
        try
        {
            obj1.display();
        }
        catch (RuntimeException e)
        {
            System.out.println("Runtime Exception raised");
        }
    }
}
```

In the preceding code, the statement, `throw new RuntimeException` is used to throw a `RuntimeException` object and the calling method, `main()` handles the exception raised in the `display()` method.

The `throw` keyword can also be used inside a catch block to **rethrow an exception**. Exceptions are **rethrown** if a method causing the error wants to pass the error handling responsibilities to the exception handlers in some other methods. You can use the following code snippet to rethrow an exception:

```
catch(Exception e)
{
    System.out.println("Exception Raised");
    throw e;
}
```

In the preceding code snippet, the exception goes to the catch block of the next higher context ignoring the catch blocks of the same try block. You can use the following code to catch the `RuntimeException` exception and rethrow the exception to the outer handler:

```
class RethrowException
{
    static void compute()
    {
        try
        {
            throw new RuntimeException ("My Exception");
        }
        catch(RuntimeException e)
        {
            System.out.println("Exception caught in compute() method");
            throw e; // Rethrow the Exception.
        }
    }
}

public static void main(String args[])
{
    try
    {
        compute();
    }
    catch(RuntimeException e)
    {
        System.out.println("Exception caught:" + e);
    }
}
}
```



throws

The throws keyword is used by a method to specify the types of exceptions that the method can throw. If a method is capable of raising a checked exception that it does not handle, then the method must specify that the exception is handled by the calling method. This is done by using the throws keyword.

Consider the following code that demonstrates the implementation of the throws statement:

```
public class ThrowsDemo
{
    void display() throws Exception
    {
        throw new Exception();
    }

    public static void main(String[] args) {
        ThrowsDemo obj1 = new ThrowsDemo();
        try
        {
            obj1.display();
        }
        catch (Exception e)
        {
            System.out.println("Runtime Exception raised");
        }
    }
}
```

In the preceding code, the display() method declares that an Exception class exception can be generated. Therefore, the try-catch blocks have been defined to catch the exception in the main() method.

finally



During the execution of a Java program, when an exception is raised, the rest of the statements in the try block are ignored. Sometimes, it is necessary to execute certain statements irrespective of whether an exception is raised. The finally block is used to execute these required statements. The statements specified in the finally block are executed after the control has left the try-catch block.

You can use the following syntax to declare the try and finally block:

```
try
{
// Block of code
}
finally
{
// Block of code that is always executed irrespective of an
exception being raised.
}
```

In the preceding syntax, the try and the finally blocks are declared.

Control Flow inside try-catch() finally:

```
try
{
    Java Statement-1;
    Java Statement-2;
    Java Statement-3;
}
catch(exceptionname obj)
{
    Java Statement-4;
}
Finally
{
    java Statement-5
}
```

Java Statement-6;

Case 1: If there is no exception raised

Case 2: If an exception raised at Java Statement-2 and corresponding catch block matched

Case 3: If an exception raised at Java Statement-2 and corresponding catch block not matched

Case 4: If an exception raised in Java Statement-4

Case 5: If an exception raised at Java Statement-5 or Java Statement-6.

If there is a catch block associated with the try block, the finally block is written after the catch block. The following code snippet shows how to declare the try, catch, and finally blocks:

```
try
{
    // Block of code.
}
catch(exceptionname1 obj1)
{
    System.out.println("Exception1 has been raised");
}
catch(exceptionname2 obj2)
{
    System.out.println("Exception2 has been raised");
}
finally
{
    // Block of code that is always executed irrespective of an exception
    being raised or not.
}
```

In the preceding code snippet, the try, catch, and the finally block are declared.

The finally block executes irrespective of whether or not an exception is raised. If an exception is raised, the finally block executes even if none of the catch blocks match the exception.

Java finally Example

```
public class FinallyExample
{
    public static void main(String args[])
    {
        try
        {
            System.out.println("Inside try block");
            // below code throws divide by zero exception
            int data=25/0;
            System.out.println(data);
        }
        // handles the Arithmetic Exception / Divide by zero exception
        catch (ArithmeticException e)
        {
            System.out.println("Exception handled");
            System.out.println(e);
        }
        // executes regardless of exception occurred or not
        finally
        {
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    }
}
```

User-defined Exceptions

In addition to the built-in exceptions, you can create customized exceptions, as per the application requirements. To create a user-defined exception, you need to perform the following steps:

1. Create an exception class.
2. Implement user-defined exception.

Creating an Exception Class

If you want to create a new user-defined exception, then the class should extend the Throwable class or its subclasses. Consider the following code snippet:

```
public class AgeException extends RuntimeException
{
    public AgeException()
    {
        System.out.println("Invalid value for age");
    }
    AgeException(String msg)
    {
        super(msg);
    }
}
```

Implementing User-defined Exception

Consider the following code to demonstrate the implementation of a user-defined exception:

```
import java.util.*;
public class ValidateAge
{
    public static void main(String[] args)
    {
        int age;
        Scanner obj1 = new Scanner(System.in);
        System.out.println("Enter the age: ");
        age = obj1.nextInt();
    }
}
```

```
    if (age <= 0)
    {
        try
        {
            throw new AgeException();
        }
        catch (AgeException e)
        {
            System.out.println("Exception raised");
        }
    }
    else
    {
        System.out.println("Age entered is " + age);
    }
}
}
```

In the preceding code, if the entered age is less than or equal to 0, the user-defined exception, `AgeException` is thrown. When the exception is thrown, the constructor defined in the class of the user-defined exception is invoked and the Invalid value for age message is displayed.

For example, when the age is specified as 12, the following output is displayed:

Age entered is 12

Difference between final, finally and finalize

Sr. no.	Key	final	finally	finalize
1.	Definition	final is the keyword and access modifier which is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java which is used to perform clean up processing just before object is garbage collected.
2.	Applicable to	Final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
3.	Functionality	(1) Once declared, final variable becomes constant and cannot be modified. (2) final method cannot be overridden by sub class. (3) final class cannot be inherited.	(1) finally block runs the important code even if exception occurs or not. (2) finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
4.	Execution	Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. It's execution is not dependant on the exception.	finalize method is executed just before the object is destroyed.

Java finalize Example

```
public class FinalizeExample {  
    public static void main(String[] args)  
    {  
        FinalizeExample obj = new FinalizeExample();  
  
        // printing the hashcode  
        System.out.println("Hashcode is: " + obj.hashCode());  
  
        obj = null;  
        // calling the garbage collector using gc()  
  
        System.gc();  
        System.out.println("End of the garbage collection");  
    }  
    // defining the finalize method  
    protected void finalize()  
    {  
        System.out.println("Called the finalize() method");  
    }  
}
```

Nested try catch block in Java

- When a try catch block is present in another try block then it is called the nested try catch block. Each time a try block does not have a catch handler for a particular exception, then the catch blocks of parent try block are inspected for that exception, if match is found that that catch block executes.
- If neither catch block nor parent catch block handles exception then the system generated message would be shown for the exception, similar to what we see when we don't handle exception.

```
class NestedTryCatch{
    public static void main(String args[]){
        //Parent try block
        try{
            //Child try block1
            try{
                System.out.println("Inside block1");
                int b =45/0;
                System.out.println(b);
            }
            catch(ArithmeticException e1){
                System.out.println("Exception: e1");
            }
            //Child try block2
            try{
                System.out.println("Inside block2");
                int b =45/0;
                System.out.println(b);
            }
            catch(ArrayIndexOutOfBoundsException e2){
                System.out.println("Exception: e2");
            }
            System.out.println("Just other statement");
        }
        catch(ArithmeticException e3){
            System.out.println("Arithmetic Exception");
            System.out.println("Inside parent try catch block");
        }
        catch(ArrayIndexOutOfBoundsException e4){
            System.out.println("ArrayIndexOutOfBoundsException");
            System.out.println("Inside parent try catch block");
        }
        catch(Exception e5){
            System.out.println("Exception");
            System.out.println("Inside parent try catch block");
        }
        System.out.println("Next statement..");
    }
}
```

Working with Threads

- Most computer games use graphics and sounds. The graphics, score, and audio effects are presented simultaneously. Imagine a computer game where you first see the screen changing, then you see your scores that are being updated, and finally, you hear the sound effects.
- In order to appreciate and enjoy a game, all these elements need to be processed simultaneously. In other words, the program should be able to efficiently perform three tasks simultaneously.
- In Java, you can achieve the preceding functionality by making use of *threads*. The game can be divided into three subunits that can be executed in parallel by using individual threads.



Thread in JAVA

- A Thread, is the path followed when executing a program.
- All Java programs have at least one thread, known as the main thread, which is created by the Java Virtual Machine (JVM) at the program's start, when the main() method is invoked.
- In Java, creating a thread is accomplished by implementing an `Runnable` interface or extending a `Thread` class.
- Every Java thread is created and controlled by the `java.lang.Thread` class.

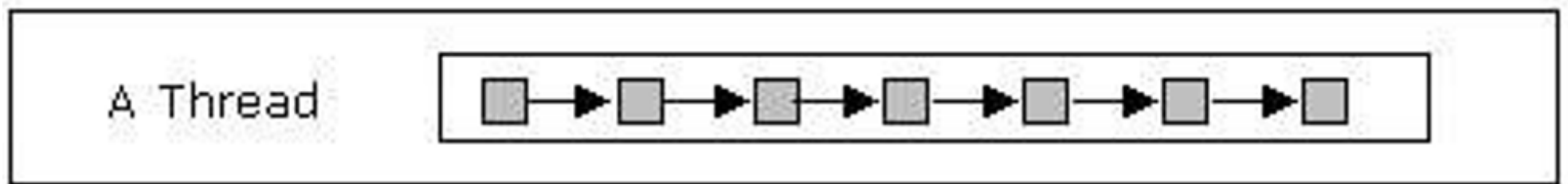


Example

- For example, a Central Processing Unit (CPU) performs various processes simultaneously, such as writing and printing a document, installing software, and displaying the date and time. All these processes are handled by separate threads.
- An application comprises one or more processes that can be performed simultaneously. Further, each process is broken into small chunks known as tasks.
- Each thread is assigned a unique task to perform, independently. For example, a text editor is an application that allows you to perform two tasks, such as write to a document and print a document, simultaneously.

The Basic Concept of Multithreading

A single-threaded application can perform only one task at a time. The following figure shows a single-threaded application.

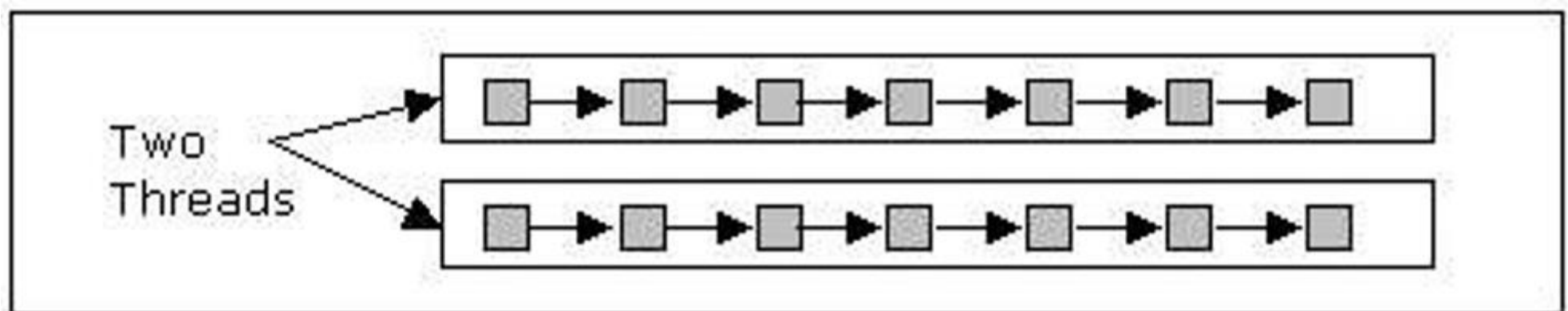


A Single-threaded Application

A single-threaded application needs to wait for one task to complete before another can start. As a result, the efficiency of the application is hampered. On the other hand, you can apply multithreading when there is a requirement to enable an application to perform several tasks simultaneously.



Multithreading helps to perform multiple tasks simultaneously, which saves time and improves the execution speed and efficiency of the application. Every application has at least one thread and you can create more threads, if required. The following figure shows a multi-threaded application.



A Multi-threaded Application

The microprocessor allocates the memory to the processes that are executed by an application and each process occupies its own address space or memory. However, all the threads in a process occupy the same address space.



Advantages and Disadvantages of Multithreading

Most of the computers today utilize software and hardware, such as operating system and multiple processors that support multithreading. Therefore, to achieve efficiency and ensure an optimized performance, most of the applications utilize multithreading. However, there are certain advantages and disadvantages of using multithreading that can be identified. The various advantages of multithreading are:

- ❑ **Improved performance:** Provides improvement in the performance of the processor by the simultaneous execution of computation and Input/Output (I/O) operations.



- ❑ **Minimized system resource usage:** Minimizes the use of system resources by using threads, which share the same address space and belong to the same process.
- ❑ **Program structure simplification:** Simplifies the structure of complex applications, such as multimedia applications. Subprograms can be written for each activity that makes a complex program easy to design and code.

Disadvantages of multithreading

- ❑ **Race condition:** When two or more threads simultaneously access the same variable and at least one thread tries to write a value to the variable, it is called the race condition. This condition is caused when synchronization between the two threads is not implemented. For example, in a word processor program, there are two threads, ThreadA and ThreadB. ThreadA wants to read a file, and at the same time, ThreadB wants to write to a file. If ThreadA reads the value before ThreadB performs a write operation, ThreadA will fail to receive the updated value from the file. Such a condition is known as a race condition.
- ❑ **Deadlock condition:** The *deadlock* condition arises in a computer system when two threads wait for each other to complete the operations before performing the individual action. As a result, the two threads are locked and the program fails. For example, consider two threads, ThreadA and ThreadB. ThreadA is waiting for a lock to be released by ThreadB, and ThreadB is also waiting for the lock to be released by ThreadA to complete its transaction. This situation is a deadlock condition.



- ❑ **Lock starvation:** *Lock starvation* occurs when the execution of a thread is postponed because of its low priority. JRE executes threads based on their priority because the CPU can execute only one thread at a time. The thread with a higher priority is executed before the thread with a lower priority, and therefore, there is a possibility that the execution of the thread that has a lower priority is always postponed.

The Thread Class

The `java.lang.Thread` class is used to construct and access the individual threads in a multi-threaded application. You can create a multi-threaded application by using the Thread class or the Runnable interface.

The `Thread` class contains various methods that can be used to work with a thread, such as setting and checking the properties of a thread, causing a thread to wait, and being interrupted or destroyed. You can make the classes run in separate threads by extending the Thread class. Some of the methods defined in the Thread class are:

- ❑ `int getPriority():` Returns the priority of a thread.
- ❑ `boolean isAlive():` Determines whether a thread is running.

- `static void sleep(long milliseconds)`: Causes the thread execution to pause for a period of time as specified by the value of milliseconds.
- `String getName()`: Returns the name of a thread.
- `void start()`: Starts a thread execution by calling the `run()` method.
- `static Thread currentThread()`: Returns a reference to the thread object that is currently executing.
- `boolean isAlive()`: Is used to check the existence of a thread. This method returns true if the thread is running, and returns false if the thread is new or gets terminated.
- `public final void join()`: Allows a thread to wait until the thread, on which it is called, terminates.

- **wait():** Informs the current thread to leave its control over the monitor and wait until another thread calls the notify() method.
- **notify():** Wakes up a single thread that is waiting for the monitor of the object being executed. If multiple threads are waiting, one of them is chosen randomly.
- **notifyAll():** Wakes up all the threads that are waiting for the monitor of the object.
- **void interrupt():** Is used to interrupt the execution of a thread.
- **static boolean interrupted():** Is used to determine if the current thread has been interrupted by another thread.
- **static void yield():** it can stop the currently executing thread and will give a chance to other waiting threads of the same priority. If in case there are no waiting threads or if all the waiting threads have low priority then the same thread will continue its execution.

You can use the following code to understand the usage of the methods in the Thread class:



```
class MainThreadDemo
{
    public static void main(String args[])
    {
        Thread t= Thread.currentThread();
        System.out.println(" The current thread: " + t);
        t.setName("MainThread");
        System.out.println(" The current thread after name change : "
+ t);
        System.out.println(" The current Thread is going to sleep for
10 seconds");
        try
        {
            t.sleep(10000);
        }
        catch(InterruptedException e)
        {
            System.out.println("Main thread interrupted");
        }
        System.out.println(" After 10 seconds.....the current Thread
is exiting now.");
    }
}
```

The Life Cycle of a Thread

A thread is a path of execution in a program that enters any one of the following five states during its life cycle. The five states are as follows:

- ☐ New
- ☐ Runnable
- ☐ Running
- ☐ Blocked (Not-Runnable State)
- ☐ Terminated or Dead

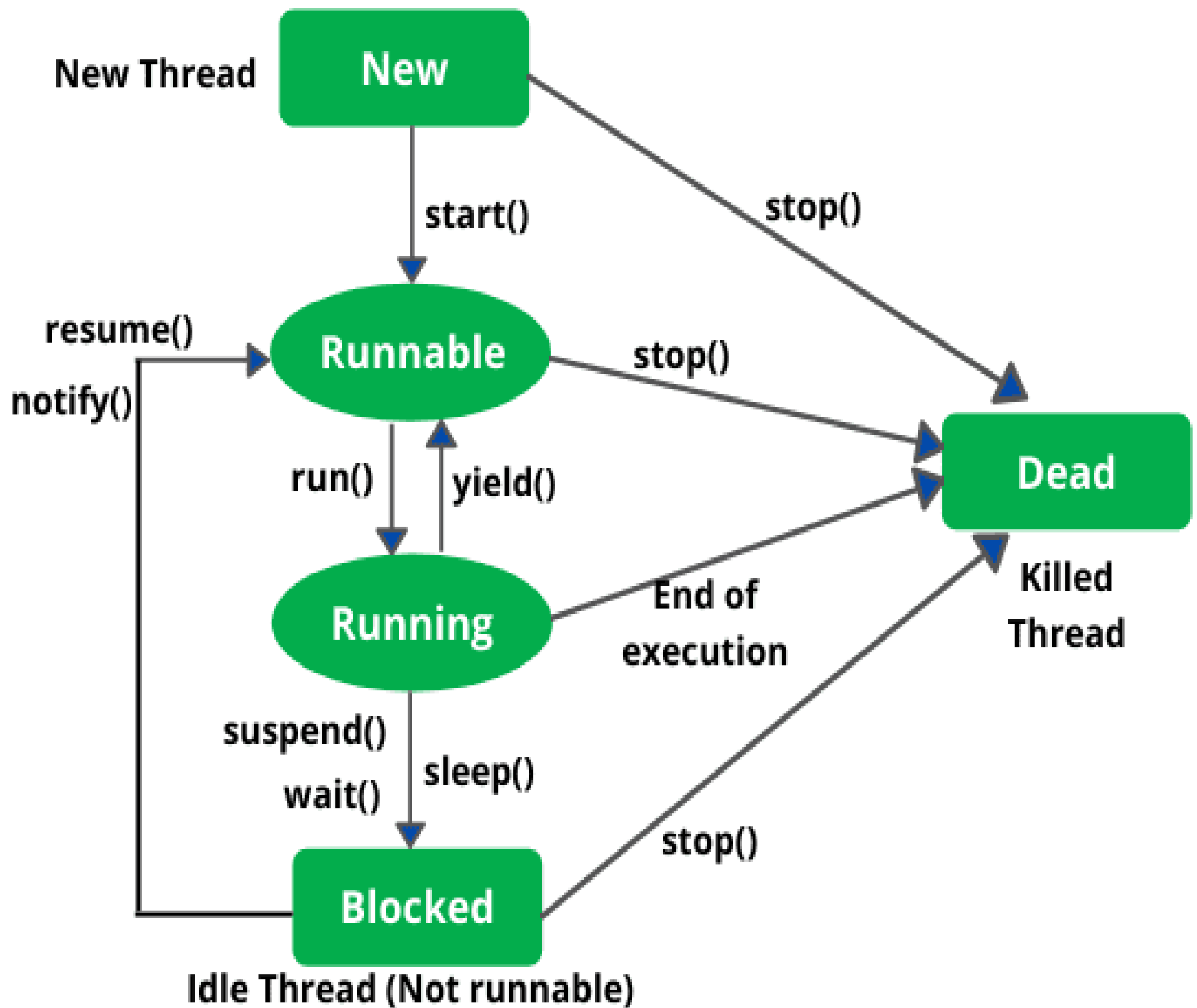
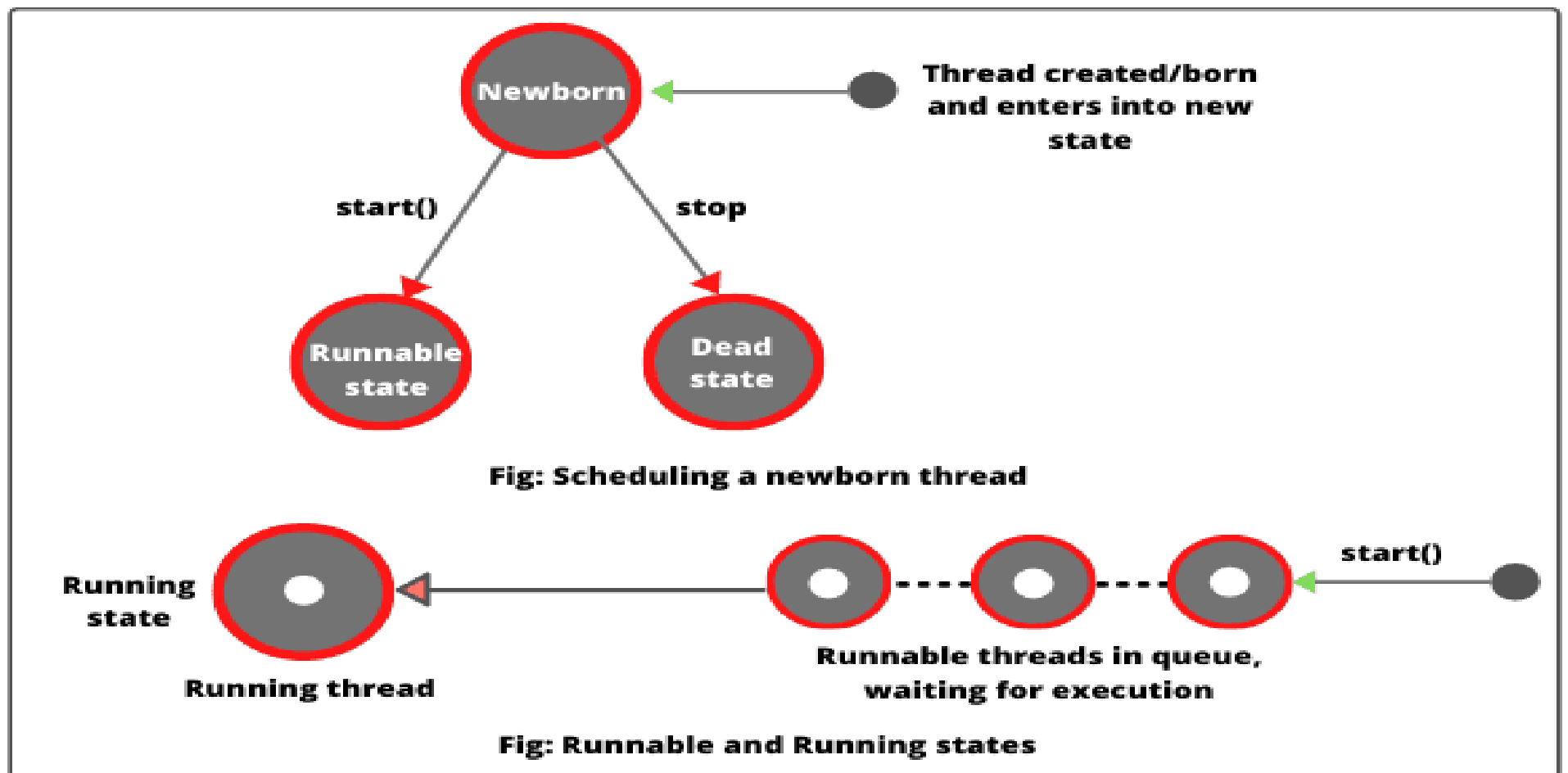


Fig: State Transition Diagram of a Thread

1. New (Newborn State)

1. New State: When we create a thread object using Thread class, thread is born and is known to be in Newborn state. That is, when a thread is born, it enters into **new state** but the start() method has not been called yet on the instance.



In other words, Thread object exists but it cannot execute any statement because it is not an execution of thread. Only start() method can be called on a new thread; otherwise, an **IllegalThreadStateException** will be thrown.



2. Runnable state

Runnable state means a thread is ready for execution. When the `start()` method is called on a new thread, thread enters into a runnable state.

In runnable state, thread is ready for execution and is waiting for availability of the processor (CPU time). That is, thread has joined queue (line) of threads that are waiting for execution.

If all threads have equal priority, CPU allocates time slots for thread execution on the basis of first-come, first-serve manner. The process of allocating time to threads is known as **time slicing**. A thread can come into runnable state from running, waiting, or new states.



3. Running state

Running means Processor (CPU) has allocated time slot to thread for its execution. When thread scheduler selects a thread from the runnable state for execution, it goes into running state.

In running state, processor gives its time to the thread for execution and executes its **run** method. This is the state where thread performs its actual functions. A thread can come into running state only from runnable state.

A running thread may give up its control in any one of the following situations and can enter into the blocked state.

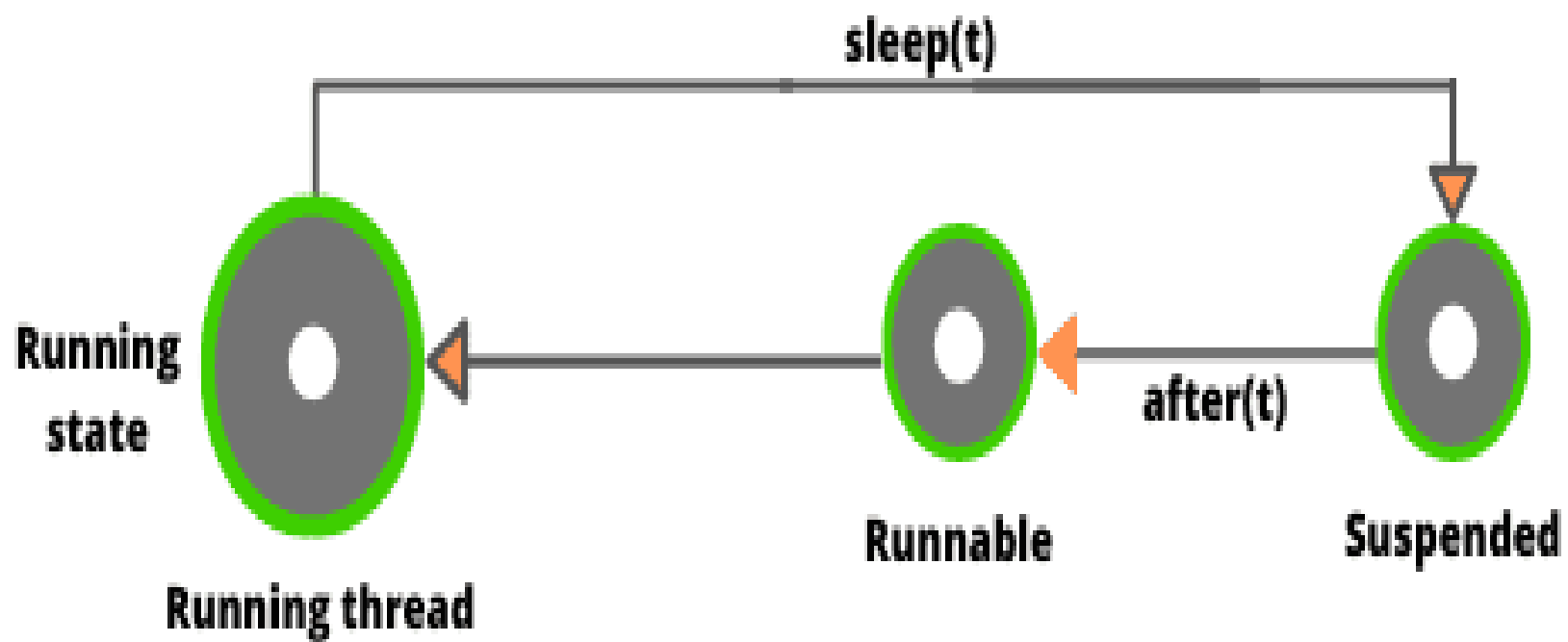


Fig: Relinquishing control using sleep() method

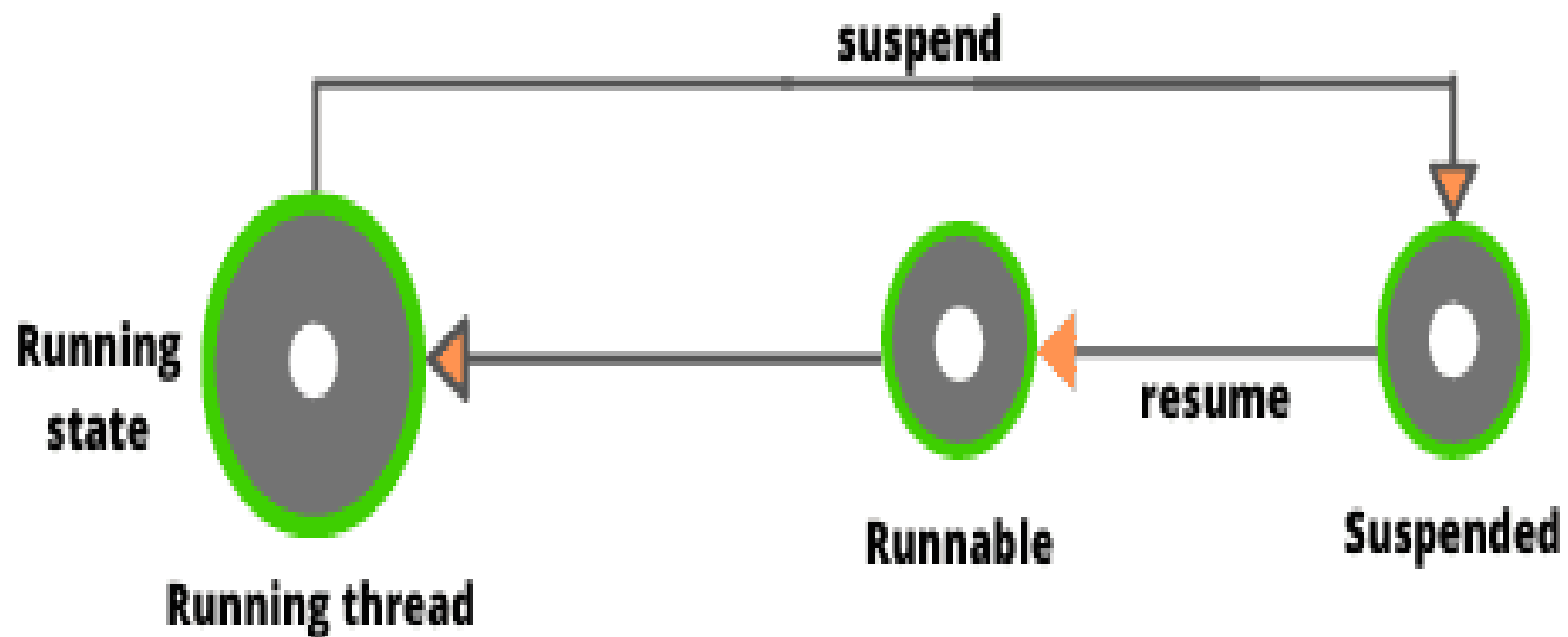


Fig: Relinquishing control using suspend() method



- a) When `sleep()` method is invoked on a thread to sleep for specified time period, the thread is out of queue during this time period. The thread again reenters into the runnable state as soon as this time period is elapsed.
- b) When a thread is suspended using `suspend()` method for some time in order to satisfy some conditions. A suspended thread can be revived by using `resume()` method.
- c) When `wait()` method is called on a thread to wait for some time. The thread in wait state can be run again using `notify()` or `notifyAll()` method.

4. Blocked state

A thread is considered to be in the blocked state when it is **suspended, sleeping, or waiting** for some time in order to satisfy some condition.

5. Dead state

- A thread enters the dead state once the statements in the `run()` method have executed. Assigning a null value to a thread object also changes the state of the thread to dead.
- The `isAlive()` method of the Thread class is used to determine whether a thread is alive or not. You cannot restart a dead thread.
- In addition, a thread can be killed by stopping its execution with the help of the `stop()` method.

Creating Threads

Consider a scenario of a jumbled up word game where you need to implement a timer in the game. The timer will be used to provide a duration of 60 seconds within which a player has to find the correct word corresponding to a jumbled up word. Further, the timer will be displayed on the game screen when a player is playing the game. When the timer reaches 0, the "Time Up!!!" message will be displayed to the player and the game will stop.

In order to implement the preceding functionalities, you can create a thread. This can be done by instantiating an object of the Thread class. You can create a thread in a Java application by using the following approaches:

- ☐ By extending the **Thread** class
- ☐ By implementing the **Runnable** interface

Creating a Thread by Extending the Thread Class

You can create threads by extending the Thread class. You need to specify the task to be performed by a thread in the overridden `run()` method.

You can use the following code to create a thread to implement the timer by extending the Thread class:

```
public class CountdownTimer extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```



```
        for (int i = 60; i >= 0; i--)
```

```
        {
```

```
            try
```

```
            {
```

```
                Thread.sleep(1000);
```

```
                System.out.println("        "+ i + " seconds to go..");
```

```
            }
```

```
            catch (Exception e)
```

```
            {
```

```
                System.out.println(e);
```

```
            }
```

```
        }
```

```
        System.out.println("Time Up !!!!");
```

```
    }
```

```
    public static void main(String args[])
```

```
    {
```

```
        CountdownTimer obj = new CountdownTimer();
```

```
        obj.start();
```

```
    }
```

```
}
```

Creating a Thread by Implementing the Runnable Interface

Consider a scenario where you need to implement threads in a Java application and also extend another class to use its functionality, such as JFrame. Because Java does not support multiple inheritance, it provides the Runnable interface to solve this problem.

The Runnable interface only consists of the run() method, which is executed when the thread is started. When a program needs to inherit from a class other than the Thread class, you need to implement the Runnable interface to implement threads. The signature of the run() method is:

```
public void run()
```

The run() method contains the code that the new thread will execute. When the run() method is called, another thread starts executing concurrently with the main thread in the program.


```
public class CountdownTimerRunnable implements Runnable
{
    public void run()
    {
        for (int i = 60; i >= 0; i--)
        {
            try
            {
                Thread.sleep(1000);
                System.out.println("      "+ i + " seconds to go..");
            }
            catch (Exception e)
            {
                System.out.println(e);
            }
        }
        System.out.println("Time Up !!!!");
    }

    public static void main(String args[])
    {
        CountdownTimerRunnable obj = new CountdownTimerRunnable();
        Thread t=new Thread(obj);
        t.start();
    }
}
```

How to Stop Thread in Java

A thread in Java program will terminate (or move to dead state) automatically when it comes out of run() method.

But if we want to stop a thread from running or runnable state, we will need to calling stop() method of Thread class. The stop() method is generally used when we desire premature death of a thread.

The general syntax for calling stop method in java programming is as follows:

```
static void stop()
```

```
class Thread1 implements Runnable
```

```
{
public void run()
{
    System.out.println("First child thread");
}
}
```



```
class Thread2 implements Runnable
```

```
{
    static Thread t2;
    public void run()
    {
        for(int i = 0; i <= 10; i ++)
        {
            System.out.println("Second child thread: " +i);
            if(i==5)
            {
                t2.stop(); // Calling stop() method to kill running thread.
            }
        }
    }
}
```

```
public class MyThreadClass
{
```

```
    public static void main(String[] args)
    {
        Thread1 th1 = new Thread1();
        Thread2 th2 = new Thread2();

        Thread t1 = new Thread(th1);
        Thread t2 = new Thread(th2);

        t1.start();
        t1.stop(); // Calling stop() method to kill runnable thread.
        t2.start();
    }
}
```

Blocked state of a Thread

A thread moves to the blocked state when it wants to access an object that another thread has locked. Once that resource is available (unlocked) for the thread, it is no longer blocked and moves to the runnable state.

For example, if a thread tries to access a shared resource that is currently being used by another thread, it will be blocked until that resource becomes available.

Waiting state

A thread is in a waiting state when it is waiting for a specific event or a signal from another object. The thread will remain in this state until another object send the signal using `notify()` or `notifyAll()`. Once this signal is received, it becomes runnable.

Using the `isAlive()` and `Join()` Method

Sometimes, in a multithreaded program, there may be a requirement to ensure that the execution of the main thread finishes after all the other threads have executed.

For this, they will need to know whether a thread has finished its execution or else they can join the execution of a thread with the main() method.

This can be determined by using the `isAlive()` method and the `join()` method.



class NewThreadClass implements Runnable

```
{
    Thread t;
    NewThreadClass()
    {
        t = new Thread(this,"ChildThread" );
        System.out.println("Thread created: " + t);
        t.start();
    }
    public void run()
    {
        try
        {
            for(int i=1;i<=5;i++)
            {
                System.out.println(t + "loop :" + i); Thread.sleep(100);
            }
        }
        catch( InterruptedException obj)
        {
            System.out.println("Thread :" + t + "interrupted");
        }
    }
}
```




```
class IsAliveDemo
{
    public static void main(String args[])
    {
        NewThreadClass obj = new NewThreadClass();
        System.out.println(obj.t + "is alive ? : " +obj.t.isAlive());

        try
        {
            for(int i=1;i<=5;i++)
            {
                System.out.println("Main Thread loop:" + i);
                Thread.sleep(200);
            }
        }
        catch(InterruptedException e)
        {
            System.out.println("Main thread is interrupted");
        }
        System.out.println(obj.t + "is alive ? : " +obj.t.isAlive());
        System.out.println("Main Thread is exiting");
    }
}
```

```
E:\Cimage\PPT\Java Core\Thread>java IsAliveDemo
Thread created: Thread[ChildThread,5,main]
Thread[ChildThread,5,main]is alive ? : true
Main Thread loop:1
Thread[ChildThread,5,main]loop :1
Thread[ChildThread,5,main]loop :2
Main Thread loop:2
Thread[ChildThread,5,main]loop :3
Thread[ChildThread,5,main]loop :4
Main Thread loop:3
Thread[ChildThread,5,main]loop :5
Main Thread loop:4
Main Thread loop:5
Thread[ChildThread,5,]is alive ? : false
Main Thread is exiting
```

Using the join() Method

The join() method allows a thread to wait until the thread, on which it is called, terminates. In addition, the join() method enables you to specify the maximum amount of time that you need to wait for the specified thread to terminate. The signature of the join() method is:

```
public final void join() throws InterruptedException
```

In the preceding signature, if another thread interrupts, the join() method throws the InterruptedException exception. The execution of a thread can be interrupted by another thread by using the interrupt() method of the Thread class.

To determine if the current thread has been interrupted by another thread, the Thread class provides the interrupted() method.

Identifying the Thread Priorities

JRE executes threads based on their priority. Since a CPU is able to execute only one thread at a time, the threads that are ready for execution are scheduled in a queue to be executed by the processor.

The threads are scheduled by JRE by using fixed-priority scheduling. Each thread has a priority that affects its position in the thread queue of the processor.

A thread with a higher priority is scheduled for execution before the threads with a lower priority.

Defining Thread Priority

Thread priorities are integers in the range of 1 to 10 that specify the priority of one thread with respect to the priority of another thread. The execution of multiple threads on a single CPU in a specified order is called scheduling.

If the processor encounters another thread with a higher priority, the current thread is pushed back and the thread with the higher priority is executed. The next thread of a lower priority starts executing if a higher priority thread stops or becomes not runnable.

A thread is pushed back in the queue by another thread if it is waiting for an I/O operation. A thread can also be pushed back in the queue when the time for which the `sleep()` method was called on another higher priority thread gets over.

Setting the Thread Priority

Once a thread is created, its priority is set by using the `setPriority()` method of the `Thread` class. The signature of the `setPriority()` method is:

```
public final void setPriority(int newPriority)
```

In the preceding signature, the `newPriority` parameter specifies the new priority setting for a thread. The priority levels should be within the range of two constants, `MIN_PRIORITY` and `MAX_PRIORITY`.

The `MIN_PRIORITY` and `MAX_PRIORITY` constants have the values 1 and 10, respectively.

The `setPriority()` method throws the `IllegalArgumentException` exception if the priority level is less than `MIN_PRIORITY` and greater than `MAX_PRIORITY`.

The thread can be set to a default priority by specifying the `NORM_PRIORITY` constant in the `setPriority()` method.

`MAX_PRIORITY` is the highest priority that a thread can have. `MIN_PRIORITY` is the lowest priority a thread can have, and `NORM_PRIORITY` is the default priority set for a thread.

class ChildThread implements Runnable

{

Thread t;

ChildThread(int p)

{

t = new Thread(this, "ChildThread");

t.setPriority(p);

System.out.println("Thread created: " + t);

}

public void run()

{

try

{

for(int i=1;i<=5;i++)

{

System.out.println(t + "loop :" + i);

Thread.sleep(500);

}

}

catch(InterruptedException obj)

{

System.out.println("Thread :" + t + "interrupted");

}

}

}

```
class PriorityDemo
{
    public static void main(String args[])
    {
        ChildThread obj1 = new ChildThread(Thread.NORM_PRIORITY - 2);
        ChildThread obj2 = new ChildThread(Thread.NORM_PRIORITY + 2);
        ChildThread obj3 = new ChildThread(Thread.NORM_PRIORITY + 3);

        //Starting the threads with different priority obj1.t.start();
        obj2.t.start();
        obj3.t.start();

        try
        {
            System.out.println("Main thread waiting for child thread to finish");
            obj1.t.join();
            obj2.t.join();
            obj3.t.join();
        }
        catch (InterruptedException e)
        {
            System.out.println("Main thread is interrupted");
        }

        System.out.println(obj1.t + "is alive ? : " + obj1.t.isAlive());
        System.out.println(obj2.t + "is alive ? : " + obj2.t.isAlive());
        System.out.println(obj3.t + "is alive ? : " + obj3.t.isAlive());
        System.out.println("Main Thread is exiting");
    }
}
```

```
E:\Cimage\PPT\Java Core\Thread>java PriorityDemo
Thread created: Thread[ChildThread,3,main]
Thread created: Thread[ChildThread,7,main]
Thread created: Thread[ChildThread,8,main]
Main thread waiting for child thread to finish
Thread[ChildThread,8,main]loop :1
Thread[ChildThread,7,main]loop :1
Thread[ChildThread,8,main]loop :2
Thread[ChildThread,7,main]loop :2
Thread[ChildThread,8,main]loop :3
Thread[ChildThread,7,main]loop :3
Thread[ChildThread,8,main]loop :4
Thread[ChildThread,7,main]loop :4
Thread[ChildThread,8,main]loop :5
Thread[ChildThread,7,main]loop :5
Thread[ChildThread,3,main]is alive ? : false
Thread[ChildThread,7,]is alive ? : false
Thread[ChildThread,8,]is alive ? : false
Main Thread is exiting
```

Synchronization in Java

Consider the scenario of a banking application that allows you to access a bank account and perform operations, such as deposit and withdraw amount. In addition, these operations can be performed by using mediums, such as the Internet and Automated Teller Machine (ATM). In such a scenario, there is a possibility that multiple operations are performed on an account simultaneously. This can lead to loss of data or inconsistency in data.

Therefore, to avoid such issues, you need to implement a mechanism using which only a single operation can be performed on an account at a time. For this, you can apply thread synchronization.

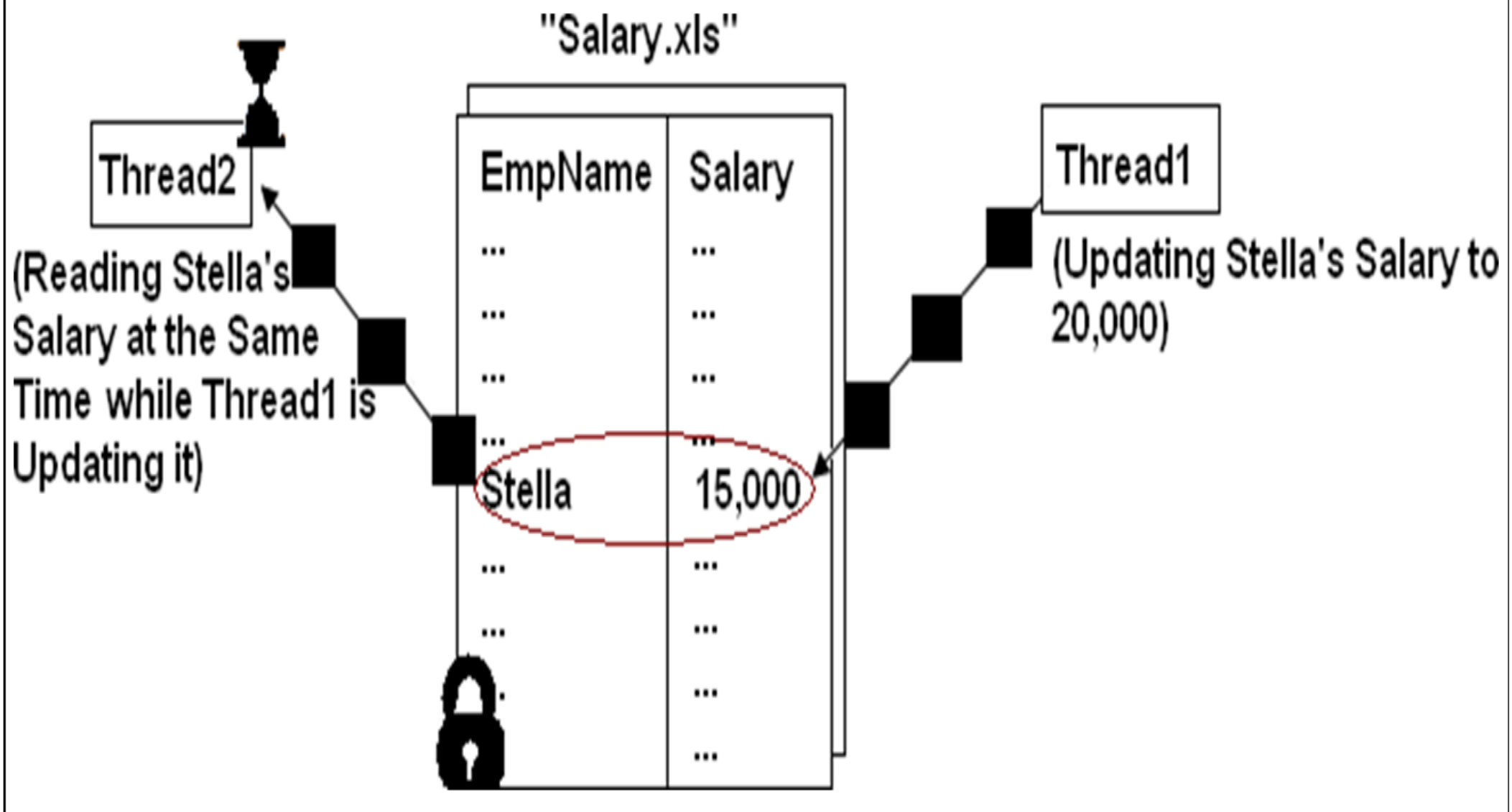
The synchronization of threads ensures that if two or more threads need to access a shared resource, then that resource is used by only one thread at a time. You can implement thread synchronization by using the following approaches:

- ❑ By using the synchronized methods
- ❑ By using the synchronized statement

The Synchronized Methods

You can synchronize a method by using the synchronized keyword. Synchronization is based on the concept of a monitor, which is an object used as a lock. All the Java objects have a monitor and only one thread can hold the monitor of an object at a given time. To enter an object's monitor, you need to call a synchronized method. The monitor controls the way in which synchronized methods access an object or class. When a thread acquires a lock, it is said to have entered the monitor. The monitor ensures that only one thread has access to the resources at any given time.

When a thread is within a synchronized method, all the other threads that try to call the method at the same time have to wait. During the execution of a synchronized method, the object is locked so that no other synchronized method can be invoked. The monitor is automatically released when the method completes its execution. The following figure shows an example of thread synchronization.



An Example of Thread Synchronization

In the preceding figure, Thread1 has a lock on the Salary.xls file, while updating the salary of Stella in the file. Thread2 wants to read the salary of Stella from the Salary.xls file and is waiting for Thread1 to release the lock on the file. While a thread is sleeping or waiting, it temporarily releases the lock it holds.



The following code snippet demonstrates how to implement synchronization for the preceding scenario by using a synchronized method:

```
class EmployeeDetails
{
    // Details of the employee
}
class EmployeeSalaryTransaction
{
    public synchronized int processSalary()
    {
        // Code to process the salary
    }
}
```

In the preceding code snippet, the `processSalary()` method is a synchronized method. This method cannot be accessed by multiple threads at a time.

The Synchronized Statement

Synchronization among threads can also be achieved by using synchronized statements. The synchronized statement is used where the synchronization methods are not used in a class and you do not have access to the source code.

Further, when a method contains only a small section of critical code for which synchronization is required, then you can use the synchronized statement instead of synchronizing the entire method.

You can synchronize the access to an object of this class by placing the calls to the methods defined by it inside a synchronized block. The following code snippet shows how to use the synchronized block:

```
synchronized(obj)
{
    /* statements to be synchronized */
}
```



In the preceding code snippet, obj is a reference of the object to be synchronized.

The following code snippet demonstrates how to implement synchronization, in case of the banking application, by using the synchronized statement:

```
class EmployeeDetails
{
    // Details of the employee
}
class EmployeeSalaryTransaction
{
    public int processSalary()
    {
        synchronized(obj)
        {
            // Code to process the salary
        }
    }
}
```

In the preceding code snippet, obj refers to an object of the EmployeeDetails class. The code to process the salary is specified within a synchronized block to ensure that it cannot be accessed by multiple threads at a time.

```
class Table{
void printTable(int n)
{//method not synchronized
    
    for(int i=1;i<=5;i++)
    {
        System.out.println(n*i);
        try{
            Thread.sleep(400);
        }
        catch(Exception e){System.out.println(e);}
    }
}

class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread{  
    Table t;
```



```
        MyThread2(Table t){  
            this.t=t;  
        }  
        public void run(){  
            t.printTable(100);  
        }  
    }
```

```
class TestSynchronization1{  
    public static void main(String args[]){  
        Table obj = new Table();//only one object  
        MyThread1 t1=new MyThread1(obj);  
        MyThread2 t2=new MyThread2(obj);  
        t1.start();  
        t2.start();  
    }  
}
```

```
E:\Cimage\PPT\Java Core\Thread>java TestSynchronization1
```

5

100

10

200

15

300

400

20

500

25

//example of java synchronized method

```
class Table{
    synchronized void printTable(int n){//synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n*i);
            try{
                Thread.sleep(400);
            }
            catch(Exception e){System.out.println(e);}
        }
    }
}
```

```
class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(100);
    }
}
```

```
public class TestSynchronization2
{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

```
E:\Cimage\PPT\Java Core\Thread>java TestSynchronization2
```

5

10

15

20

25

100

200

300

400

500